Vacuum Deposition Control System

Technical Reference & User Manual

CovAP Laboratory — MBE / Physical Vapor Deposition Tool

Hardware Controlled

HEAT3 Dual-Mode Heating Power Supply (Modbus RTU) • B-RAX 3400 Vacuum Gauge (RS-485) • Arduino Shutter Controller (Serial)

Software Stack

Python 3 / PyQt5 GUI • Excel recipe import/export • Live pyqtgraph plots • BEP measurement engine • Test (simulation) mode

Version: 1.0 | Last updated: 2026 | Author: CovAP Lab

Contents

1. System Overview
2. Hardware Devices & Communication Protocols
   2.1 HEAT3 Dual-Mode Heating Power Supply — Modbus RTU
   2.2 B-RAX 3400 Vacuum Gauge — InstruTech RS-485
   2.3 Arduino Shutter Controller — Serial ASCII
3. Software Architecture
   3.1 Entry Point — main.py
   3.2 GUI Application — gui_app.py
   3.3 Hardware Drivers — devices/
   3.4 Data Logger — data_logger.py
   3.5 Recipe Engine — recipe_engine.py
   3.6 Recipe Logger — recipe_logger.py
   3.7 BEP Manager — bep_manager.py
   3.8 Simulation Devices — devices/sim_devices.py
4. GUI User Guide
   4.1 Connection & Test Mode
   4.2 Live Plot Tab
   4.3 Recipe Tab
   4.4 BEP Measurement Tab
   4.5 System Log & Debug Mode
5. Recipe Reference
   5.1 Supported Actions
   5.2 Excel Format
   5.3 GOTO Loop Example
6. BEP Measurement Protocol
7. Data Output Files
8. Installation & Dependencies
9. Troubleshooting
10. HEAT3 Critical Register Map
11. B-RAX 3400 Command Reference

1. System Overview

The Vacuum Deposition Control System is a Python/PyQt5 desktop application for controlling and logging a physical vapor deposition (PVD) tool. It provides unified control of three hardware subsystems over separate serial interfaces, a multi-step recipe engine with Excel import, a BEP (Beam Equivalent Pressure) measurement workflow, and a run logger that exports time-series data to Excel workbooks.

System Block Diagram

HEAT3	B-RAX 3400	Arduino Uno
Selenium & Substrate heaters	Ion gauge + convectron pressure	In shutter (pin 10) + Se shutter (pin 7)
Modbus RTU / RS-485	*InstruTech RS-485 ASCII*	*Serial ASCII commands*
`COM3 | 57600 8N1 | Slave 1`	`COM4 | 19200 8N1 | Addr 01`	`COM5 | 9600 8N1`

✓ **Tip**
All three COM port numbers are configurable via Settings in the GUI. The defaults above match typical lab wiring — adjust as needed.

Key Capabilities

- Live pressure plot (log scale) and dual-channel temperature plots, updated every 500 ms from thread-safe deque buffers
- Recipe engine: multi-step process programs imported from Excel (.xlsx), with SET_TEMP, WAIT_TEMP, WAIT_PRESSURE, WAIT_TIME, OPEN_SHUTTER, CLOSE_SHUTTER, HEATER_ON, HEATER_OFF, GOTO (counted loop), IG_ON, IG_OFF, END
- BEP measurement: fast 20 Hz pressure sampling; BEP = max(open window) − mean(baseline); Se/In ratio reported
- Run logger: records pressure + temperature + shutter state at 1 Hz during recipe; exports 4-sheet Excel workbook with embedded charts
- Test mode: fully simulated hardware (realistic random-walk pressure, PID temperature lag) with no physical device needed
- Debug mode: shows raw HEAT3 Modbus TX/RX hex bytes in the System Log
- PyInstaller-ready: build.spec included for single-EXE Windows distribution

2. Hardware Devices & Communication Protocols

2.1 HEAT3 Dual-Mode Heating Power Supply — Modbus RTU

The HEAT3 (Prevac) is a two-channel PID-controlled heating power supply supporting resistive and electron-bombardment modes. The system drives it using Modbus RTU over RS-485 at 57600 baud. CH1 is assigned to the Selenium (Se) source; CH2 is assigned to the Substrate heater.

Connection Parameters

Item	Detail
Baud rate	57600
Data / Parity / Stop	8 N 1
Protocol	Modbus RTU (FC 0x03 read, FC 0x06 write single word)
Slave ID	1 (configurable in device menu → Communication)
Serial timeout	0.25 s per transaction
Inter-write delay	0.01 s between the two 16-bit words of an INT32
Readback delay	0.30 s after writing setpoint before verifying
Heartbeat	READ R_STATUS1 (reg 0) every 7 s to maintain MASTER right (device drops at 10 s silence)

Remote Control / MASTER Protocol (§6.2 of HEAT3 manual)

The HEAT3 has a Remote Enable mode that must be activated from the front panel first: Settings → Communication → Remote Enable → ON. Once enabled, any computer that writes 1 to register 1000 acquires MASTER rights. MASTER rights are held as long as a Modbus frame arrives within 10 seconds. The software sends a heartbeat READ to register 0 every 7 seconds (a read is sufficient and safer than a write).

> ⚠ **Note**
> If the HEAT3 drops back to local control mid-recipe, the most likely cause is: (a) Remote Enable was never turned ON from the front panel, or (b) a serial port conflict — ensure no other software has the COM port open.

Temperature Register Encoding

All temperature registers use INT32 format (two consecutive 16-bit words, MSB at address N, LSB at address N+1), with unit = Kelvin × 100 as a signed integer.

```
Raw register value = round((°C + 273.15) × 100)
°C from raw      = raw / 100.0 − 273.15
```

Writing an INT32 requires two separate FC 0x06 writes (high word first, then low word). FC 0x10 multi-write is not used because some HEAT3 firmware versions return non-standard FC 0x10 responses.

Critical Register Map (subset used by this software)

Address	Function	Type	R/W	Notes / Values

0	CH1 Status word (bit 0 = OPERATE state)	INT16	R	bit0: 1=OPERATE, 0=STANDBY
2	CH1 Process Value (current temperature)	INT32	R	Kelvin × 100. Two words: regs 2–3
14	CH1 PID Setpoint (T mode)	INT32	R/W	Kelvin × 100. Write: regs 14, 15
21	CH1 Work Mode	UINT8	R/W	0=Manual, 1=Auto, 2=External, 3=PID Out
70	CH1 Go to OPERATE	UINT8	W	Write 1 to enter OPERATE state
71	CH1 Go to STANDBY	UINT8	W	Write 1 to enter STANDBY state
72	CH1 PID RUN	UINT8	W	Write 1 to start PID regulation
84	CH2 Status word (bit 0 = OPERATE state)	INT16	R	Same bit mapping as CH1
88	CH2 Process Value	INT32	R	Kelvin × 100. Regs 88–89
90	CH2 PID Setpoint	INT32	R/W	Kelvin × 100. Regs 90–91
97	CH2 Work Mode	UINT8	R/W	Same values as CH1 reg 21
114	CH2 Go to OPERATE	UINT8	W	Write 1
115	CH2 Go to STANDBY	UINT8	W	Write 1
116	CH2 PID RUN	UINT8	W	Write 1
151	CH1 PID Output %	INT16	R	Value × 100 = %
153	CH2 PID Output %	INT16	R	Value × 100 = %
1000	Remote Control	UINT8	R/W	Write 1 = acquire MASTER; Write 0 = release; Read = remote status
1151	Remote Enable Status	UINT8	R	1 = Remote Enable is ON

Apply Setpoint Sequence

The apply_setpoint() method in devices/heat3.py performs:

1. Write 1 → register 1000 (re-assert REMOTE MASTER)
2. Write 1 → WMODE register for channel (Auto mode)
3. Write high word of setpoint (INT32, Kelvin × 100) → SP register
4. Write low word of setpoint → SP register + 1
5. Sleep 300 ms (device latches value)
6. Read back SP register and verify within ±1°C

> ✓ **Tip**
> The batch poll_all() method reads CH1 regs 0-21 and CH2 regs 84-97 in two transactions instead of 8 individual reads, cutting poller time from ~2 s to under 0.5 s.

2.2 B-RAX 3400 Vacuum Gauge — InstruTech RS-485

The B-RAX 3400 (InstruTech) combines an ion gauge with two convectron gauges and an analog input channel. Communication uses the InstruTech RS-485 ASCII protocol.

Connection Parameters

Item	Detail
Baud rate	19200 (factory default — configurable)
Data / Parity / Stop	8 N 1
Protocol	InstruTech RS-485 ASCII (COMM Type: RS485)
Device address	01 (hex, 00–FF range, factory default = 01)
Command format	#AA<COMMAND><CR> where AA = two-hex-char address
Response format	*AA <value><CR> (13 chars total including CR)
Error response	?AA <error><CR>
Serial timeout	0.25 s
Repetition rate	Up to 46 ms at 19200 baud (from manual table)

InstruTech RS-485 Command Summary

Command / TX	Response / RX	Description
`#01RDIG<CR>`	`*01 1.53E-06<CR>`	Read ion gauge pressure in Torr. Returns 1.10E+03 when IG is off.
`#01RDCG1<CR>`	`*01 7.60E+02<CR>`	Read convectron gauge 1 pressure
`#01RDCG2<CR>`	`*01 7.60E+02<CR>`	Read convectron gauge 2 pressure
`#01RDAI<CR>`	`*01 7.60E+02<CR>`	Read analog input pressure
`#01IG1<CR>`	`*01 PROGM_OK<CR>`	Turn ion gauge ON (y=1=ON, y=0=OFF)
`#01IG0<CR>`	`*01 PROGM_OK<CR>`	Turn ion gauge OFF; also clears IG errors
`#01IGS<CR>`	`*01 1 IG_ON <CR>`	Query IG on/off status
`#01DG1<CR>`	`*01 PROGM_OK<CR>`	Turn on IG degas (requires IG on, P < 5×10^{-5} Torr)
`#01DGS<CR>`	`*01 1 DG_ON <CR>`	Query degas status
`#01RSIG<CR>`	`*01 00 ST_OK<CR>`	Read IG status (00=OK, 01=OVPRS, 02=EMISS, 08=FLOPN...)
`#01VER<CR>`	`*01 01306-11<CR>`	Read firmware version

Response Parsing

Response format: *AA_y.yyEzpp<CR> — a 13-character fixed-length string. The driver strips the leading *, skips 3 characters (2-char address + space), and converts the remainder to float. Responses starting with ? indicate invalid command or out-of-range condition and return None.

> ⚠ **Note**
> The ion gauge cannot be turned on when pressure exceeds 1×10^{-3} Torr (at 100 µA emission). Always verify chamber pressure via convectron gauge before enabling IG.

2.3 Arduino Shutter Controller — Serial ASCII

An Arduino Uno controls two relay-driven shutters: the Indium (In) source shutter on pin 10 and the Selenium (Se) source shutter on pin 7. The relays are active-LOW (writing HIGH = closed, LOW = open). The firmware (CovapDuty_v2.ino) accepts plain-text commands over Serial at 9600 baud.

Connection Parameters

Item	Detail
Baud rate	9600
Data / Parity / Stop	8 N 1
Line ending	\n (LF) after command
Response	OK:<COMMAND> on success; ERR:UNKNOWN_CMD on failure
Boot delay	2 s after serial connect (Arduino resets on DTR)

Command Reference

Command / TX	Response / RX	Description
`IN_ON`	`OK:IN_ON`	Open In shutter (pin 10 LOW)
`IN_OFF`	`OK:IN_OFF`	Close In shutter (pin 10 HIGH)
`SE_ON`	`OK:SE_ON`	Open Se shutter (pin 7 LOW)
`SE_OFF`	`OK:SE_OFF`	Close Se shutter (pin 7 HIGH)
`BOTH_ON`	`OK:BOTH_ON`	Open both shutters simultaneously
`BOTH_OFF`	`OK:BOTH_OFF`	Close both shutters simultaneously
`STATUS`	`STATUS:IN=ON,SE=OFF`	Query current shutter states

> ⚠ **Note**
> Export the updated firmware (CovapDuty_v2.ino) from the Help → Export Arduino Firmware menu item and flash it before connecting. The original CovapDuty.ino does not respond to software commands.

3. Software Architecture

3.1 Entry Point — main.py

Applies high-DPI Qt attributes, sets up Python logging (WARNING level to stderr with timestamps), instantiates QApplication, applies the dark Fusion theme, creates and shows MainWindow, and enters the Qt event loop. When frozen with PyInstaller, adds sys._MEIPASS to sys.path so bundled modules are found.

3.2 GUI Application — gui_app.py

The main window class (MainWindow) owns all device references, timers, and child components. Key design decisions:

Item	Detail
Thread-safe logging	_post_log(msg) calls _log_bridge.msg.emit(msg) on any thread. The pyqtSignal automatically marshals to the main thread. Never use QTimer.singleShot for log messages from background threads.
Hardware snapshot	Poller thread reads hardware every 1 s and writes to a Snap (thread-safe dict). The _vals() Qt timer (150 ms) reads from Snap — no serial I/O on the main thread.
Plot updates	_plots() Qt timer runs every 500 ms. When a recipe run is active, it syncs new rows from RecipeLogger into the DataLogger deque buffers using _sync_p_idx / _sync_t_idx counters to avoid duplicates.
Device resolution	_resolve() assigns self.brax / self.heat3 / self.shutter from real or sim drivers based on _test mode and whether each device is connected.
Test mode	Activating Test Mode calls _resolve() which swaps in BRAX3400Sim / HEAT3Sim / ArduinoShutterSim and auto-starts live logging.
Debug mode	Sets heat3.debug_callback to a lambda that posts TX/RX hex strings to the System Log via _post_log.

Poller Batching

For the real HEAT3, the Poller calls heat3.poll_all() which reads CH1 registers 0–21 in one FC 0x03 transaction and CH2 registers 84–97 in another, extracting status, PV, SP from each batch. This replaces 8 individual reads and halves the poller cycle time. For the simulator, individual attribute-access methods are called directly (no serial overhead).

3.3 Hardware Drivers — devices/

Item	Detail
devices/heat3.py	Real HEAT3 Modbus RTU driver. FC 0x03 reads, FC 0x06 writes. INT32 as two FC 0x06 writes. _seq_lock ensures apply_setpoint/go_operate sequences are atomic. Heartbeat thread reads R_STATUS1 every 7 s.
devices/brax3400.py	Real B-RAX 3400 ASCII driver. Builds #AA<CMD><CR> command strings, parses *AA <value><CR> responses. Thread-safe via _lock. Timeout 0.25 s.
devices/ arduino_shutter.py	Real Arduino driver. Text commands at 9600 baud. 2 s boot wait on connect. In/Se open/close/both + STATUS query. Tracks state locally.

devices/sim_devices.py	Full simulators for all three devices — see §3.8.
devices/__init__.py	Re-exports the three Sim classes for convenient importing.

3.4 Data Logger — data_logger.py

Runs a background poll thread at a configurable interval (default 1 s). Writes timestamped CSV rows (IG pressure, CH1 temp, CH2 temp) and maintains three deque buffers (pressure_buf, ch1_temp_buf, ch2_temp_buf) of (unix_time_float, value) tuples, maximum 7200 points (2 hours). The get_*_history() methods return (time_list, value_list) with t=0 at the first sample, suitable for pyqtgraph.

3.5 Recipe Engine — recipe_engine.py

Thread-based state machine with states IDLE → RUNNING → PAUSED → DONE/ERROR. The _run() loop iterates over RecipeStep objects, dispatching each to _exec(). Counted GOTO loops use a dict of remaining-count per step index. Pause/resume use a threading.Event (_pause). Key method details:

Item	Detail
start()	Clears _stop, resets loop counters, starts _run thread
pause()	Clears _pause event — _wait() calls block on it
resume()	Sets _pause event
stop()	Sets _stop, joins thread up to 3 s, sets state IDLE
_exec()	Returns 1 (advance), -1 (GOTO jump), 2 (END clean exit), None (error)
END action	Returns 2 — _run exits the while loop and falls through to set DONE state (not _stop.set() which would prevent DONE)
WAIT_TEMP	p1 = channel dropdown (Selenium/Substrate), p2 = tolerance °C. Logs PV vs SP delta every 2 s.

3.6 Recipe Logger — recipe_logger.py

Collects hardware data on a 1 Hz background thread during a recipe run. Stores lists of tuples: _rows_pressure (ts, elapsed, ig), _rows_temp (ts, elapsed, ch1, ch2), _rows_shutter (ts, elapsed, in_open, se_open). After stop(), export_excel() writes a 4-sheet workbook:

- Sheet 'Run Info': sample name, date, BEP data, recipe table, temperature chart
- Sheet 'Pressure': timestamped IG pressure + embedded line chart
- Sheet 'Temperature': CH1 (Se) and CH2 (Substrate) temp + chart
- Sheet 'Shutters': In/Se open states as 0/1 + chart

Charts use SeriesFactory(ref, title_from_data=True) — the correct openpyxl 3.1 API.

3.7 BEP Manager — bep_manager.py

Orchestrates the Beam Equivalent Pressure measurement sequence. State machine: IDLE → BASELINE → OPEN → SETTLE → BASELINE → OPEN → DONE.

Item	Detail
Baseline duration	8 s of fast (50 ms) pressure sampling with shutter closed

Shutter open duration	2 s — fast 50 ms sampling
BEP calculation	peak = max(all open-window samples); baseline = mean(all closed-window samples); BEP = peak − baseline
Mean open tracking	mean_open is also recorded in CSV for reference
Settle time	30 s between Indium and Selenium measurements
Se temperature	Read from HEAT3 CH1 at time of Se measurement
In temperature	Fixed at 744.0°C (no sensor)
CSV file	bep_log.csv — appended on each run, columns: Timestamp, Source, Temperature_C, Baseline_Mean_Torr, Peak_Max_Torr, Mean_Open_Torr, BEP_Torr
Fast buffer	self.fast_buf: list of (t_relative, pressure) from most recent open window — plotted in BEP tab Fast Trace

3.8 Simulation Devices — devices/sim_devices.py

Drop-in replacements for all real drivers used in Test Mode or when a device is not connected.

Item	Detail
BRAX3400Sim	Random-walk ion gauge 1e-9 to 5e-5 Torr (multiplicative, σ=3%). IG on/off state tracked. connected=True.
HEAT3Sim	Two _ChannelSim instances. Each has P-only PID sim with τ=120 s, 5°C/s max rate, ±0.05°C noise. Background thread ticks at 1 Hz. Implements full interface: apply_setpoint(ch,°C), go_operate(ch), go_standby(ch), ch1/ch2_read_process_value(), ch1/ch2_read_setpoint(), ch1/ch2_is_operating(), debug_callback=None.
ArduinoShutterSim	Boolean state for in_open and se_open. All commands return True. in_is_open and se_is_open properties.

4. GUI User Guide

4.1 Connection & Test Mode

On startup, all connection status LEDs (BRAX, HEAT3, Arduino) show red (disconnected).

7. Open Settings (File menu or ⚙️ button) to confirm COM port numbers and baud rates.
8. Click Connect All (or connect devices individually from the Hardware menu).
9. Status LEDs turn green when connected; version/firmware info appears in System Log.
10. To run without physical hardware: toggle 🧪 Test: OFF → Test: ON. All three sims activate immediately and live logging starts automatically.

> ⚠️ **Note**
> On first connect to the HEAT3, the device front panel must have Remote Enable set to ON (Settings → Communication → Remote Enable → ON). The software cannot enable this from software alone.

4.2 Live Plot Tab

Three time-series plots update every 500 ms. The Y axis of the pressure plot is log-scale. Start/stop CSV logging with the ▶️ Start Logging / ⏹ Stop button. The log file path is shown next to the button. When a recipe run is active, the rec_log data flows into the same plot buffers so the run is visible in real time.

Heater Controls (left panel)

Each channel (CH1 Selenium, CH2 Substrate) has: a PV display, an SP setpoint spin box, an Apply button, and an Apply result label. Clicking Apply sends apply_setpoint() and shows ✓ or ✗ with the readback temperature. The Operate/Standby buttons call go_operate() / go_standby(). In test mode, a simulated PV indicator is also shown.

4.3 Recipe Tab

Recipes are table-driven. Each row has Step number, Action dropdown, Param 1 (channel/source dropdown for relevant actions), Param 2 (numeric), and Notes columns.

Loading and Running

11. Load an Excel recipe with 📂 Load, or build one row by row with + Row.
12. Click ▶️ Run. You will be prompted for a sample name (used in the output file name).
13. The State indicator shows 🟢 RUNNING. The Step indicator shows the current step number, action, and parameters.
14. System Log shows every step as it executes with action details and results.
15. On completion (✅ DONE), the run log Excel is saved automatically. If live logging was running before the recipe, it resumes automatically after.

> ✓ **Tip**
> Pressing ⏸ pauses at the end of the current WAIT. Pressing ▶️ Run again resumes. Pressing ⏹ stops immediately and saves partial log.

4.4 BEP Measurement Tab

Click 🔬 Measure BEP to start. The sequence runs automatically:

16. Baseline measurement for Indium (8 s, shutter closed, averaging pressure)
17. Open In shutter for 2 s (20 Hz sampling)
18. Close In shutter, 30 s settle
19. Baseline measurement for Selenium (8 s)
20. Open Se shutter for 2 s (20 Hz sampling)
21. Close Se shutter; compute BEP values and ratio

Results appear in the In BEP, Se BEP, and Se/In Ratio displays. History plots (BEP vs. temperature) update after each measurement. The Fast Trace sub-tab shows the raw 20 Hz pressure trace from the most recent open window.

> ⚠️ **Note**
> BEP measurement requires both BRAX 3400 and Arduino to be connected (or Test Mode active). The prompt confirms that the ion gauge is facing the evaporation sources before starting.

4.5 System Log & Debug Mode

The System Log at the bottom of the window shows timestamped messages from all subsystems. Messages are color-coded: green for step starts, dimmed for timestamps, normal for device responses.

Enable 🐛 Debug: ON to show raw Modbus TX/RX hex bytes for every HEAT3 transaction, e.g.:

```
🐛 TX: 01 03 00 00 00 01 84 0A
🐛 RX: 01 03 02 00 00 B8 44
```

5. Recipe Reference

5.1 Supported Actions

Action	Param 1	Param 2	Behaviour
SET_TEMP	Selenium (CH1) or Substrate (CH2)	Temperature °C	Apply setpoint to selected channel via apply_setpoint()
WAIT_TEMP	Selenium (CH1) or Substrate (CH2)	Tolerance °C (default 5)	Poll PV every 2 s; wait until |PV-SP| ≤ tolerance. Logs delta each poll.
WAIT_PRESSURE	Target pressure (Torr)	—	Poll IG every 2 s; wait until P ≤ target
WAIT_TIME	Duration (seconds)	—	Sleep for specified duration, respecting pause/stop
OPEN_SHUTTER	In / Se / Both	—	Open the selected shutter(s)
CLOSE_SHUTTER	In / Se / Both	—	Close the selected shutter(s)
HEATER_ON	Selenium (CH1) or Substrate (CH2)	—	Call go_operate(ch): MASTER → Auto → OPERATE → PID RUN
HEATER_OFF	Selenium (CH1) or Substrate (CH2)	—	Call go_standby(ch)
IG_ON	—	—	Turn ion gauge on
IG_OFF	—	—	Turn ion gauge off
GOTO	Target step number	Loop count	Jump back to target step N times. 0 remaining = fall through.
END	—	—	Exit recipe cleanly, trigger DONE state and log save

5.2 Excel Format

Recipes are stored in .xlsx files with a single sheet containing a header row followed by one step per row:

Step	Action	Param1	Param2	Notes
1	HEATER_ON	Selenium (CH1)		Start Se heater
2	SET_TEMP	Selenium (CH1)	250	Set Se to 250°C
3	WAIT_TEMP	Selenium (CH1)	5	Wait ±5°C
4	OPEN_SHUTTER	Se		
5	WAIT_TIME	60		Deposition 60 s
6	CLOSE_SHUTTER	Se		
7	GOTO	4	3	Repeat 3 more times
8	HEATER_OFF	Selenium (CH1)		
9	END			

5.3 GOTO Loop Example

GOTO step 4 with loop count 3 means: the first time the engine reaches the GOTO instruction it has 3 loops remaining. It decrements to 2 and jumps to step 4. Second time: 1 remaining, jumps. Third time: 0 remaining, jumps. Fourth time: counter is gone (exhausted), falls through to the next step. Net result: steps 4-7 execute 4 times total (initial pass + 3 jumps).

6. BEP Measurement Protocol

Beam Equivalent Pressure (BEP) is the partial pressure rise caused by molecular flux from an evaporation source hitting the ion gauge. It is proportional to deposition rate and is used to set Se/In flux ratios for stoichiometric CIGSe growth.

Measurement Sequence

Phase	Duration	Action
Baseline (In)	8 s	Sample IG at 20 Hz with In shutter closed. Compute baseline_In = mean of all samples.
Open In	2 s	Open In shutter. Sample IG at 20 Hz. fast_buf_In = list of (t, P).
Close In	—	Close In shutter. peak_In = max(fast_buf_In). BEP_In = peak_In − baseline_In.
Settle	30 s	Wait for pressure to return to baseline before Se measurement.
Baseline (Se)	8 s	Sample IG at 20 Hz with Se shutter closed. baseline_Se = mean.
Open Se	2 s	Open Se shutter. fast_buf_Se recorded at 20 Hz.
Close Se	—	peak_Se = max(fast_buf_Se). BEP_Se = peak_Se − baseline_Se.
Compute ratio	—	Se/In ratio = BEP_Se / BEP_In. Stored for recipe logger.

✓ **Tip**
Using max(open window) rather than mean ensures the true peak flux is captured even if the shutter opens slowly or the gauge response time creates a sharp spike.

7. Data Output Files

Item	Detail
pressure_log_YYYYMMDD_HHMMSS.csv	Live logging CSV. Columns: Timestamp, IG_Pressure_Torr, CH1_Temp_C, CH2_Temp_C. Written in real time (line-buffered). Created when logging starts.
YYYYMMDD_HHMMSS_<sample>.xlsx	Run log Excel workbook. 4 sheets: Run Info (recipe + BEP summary + temp chart), Pressure, Temperature, Shutters. Created at end of recipe run.
bep_log.csv	BEP history CSV. Columns: Timestamp, Source, Temperature_C, Baseline_Mean_Torr, Peak_Max_Torr, Mean_Open_Torr, BEP_Torr. Appended each measurement.

8. Installation & Dependencies

Requirements

Item	Detail
Python	3.9 or later (tested on 3.11)
PyQt5	≥ 5.15.0 — GUI framework
pyqtgraph	≥ 0.13.0 — live plots (optional; falls back to a placeholder if missing)
pyserial	≥ 3.5 — serial communication
minimalmodbus	≥ 2.0.0 — not used directly but must be importable (used in earlier prototype)
openpyxl	≥ 3.0.0 — Excel recipe read/write and run log export
PyInstaller	≥ 5.0.0 — for building Windows EXE (optional)

Quick Start

22. pip install PyQt5 pyqtgraph pyserial openpyxl
23. cd vac_control
24. python main.py

Building Windows EXE

```
pip install pyinstaller
pyinstaller build.spec
```

The EXE is produced in the dist/ folder. To produce a single-file EXE, edit build.spec and set exclude_binaries=False and remove the COLLECT block.

9. Troubleshooting

Symptom	Likely Cause / Fix
HEAT3 drops REMOTE mid-recipe	(a) Remote Enable not ON on device front panel. (b) 10 s silence — verify heartbeat thread is running (check that heat3.connected=True). (c) Another process has the COM port open.
Nothing appears in System Log	Verify _log_bridge.msg is connected to _log in _wire(). In Test Mode, step messages go through on_status_msg → _post_log → signal → _log.
GUI is slow / unresponsive	HEAT3 serial timeout was 1.0 s — reduced to 0.25 s. Ensure poll_all() is being used (hasattr check in Poller). Reduce poll_interval in Settings.
Excel export crashes (TypeError: expected int)	Old code used Series(data, title_from_data=True). Fixed to use SeriesFactory(ref, title_from_data=True). Upgrade to latest code.
Recipe step indicator does not update	Step indicator uses _r_step(i) via on_step_change signal from recipe thread. Verify QTimer.singleShot wrapping is intact in _wire().
BRAX reads garbage values	Check COM port baud rate matches device (default 19200). Check response parser: strip leading *, skip 3 chars (2-char addr + space), then float().
Arduino not responding	Check firmware is CovapDuty_v2.ino (not original). Add 2+ s boot wait on connect. Verify commands are sent with \n terminator.
WAIT_TEMP never exits	Confirm heater is in OPERATE mode (HEATER_ON step before WAIT_TEMP). In test mode the sim temperature will ramp to SP automatically.
import error: No module named serial	pip install pyserial
setMaximumBlockCount error on QTextEdit	Fixed: call on document() not widget: log_w.document().setMaximumBlockCount(1000)

10. HEAT3 Critical Register Map (Full Context)

This section provides the complete register addresses used by this software plus key adjacent registers from the HEAT3 manual for context.

CH1 (Selenium Source) Registers

Address	Function	Type	R/W	Notes / Values
0	Status CH1 (bit 0 = OPERATE state, bit1 = PID autotune, bit3 = no TC1, bit4 = no TC2)	INT16	R	Bit field. Check bit0 to confirm OPERATE.
1	PID state CH1: 0=Off, 1=Run, 2=Hold	UINT8	R	
2–3	Process value CH1 (current temperature)	INT32	R	Kelvin × 100. reg2=hi, reg3=lo
4–5	Thermocouple TC1 temperature	INT32	R	Kelvin × 100
6–7	Thermocouple TC2 temperature	INT32	R	Kelvin × 100
14–15	PID setpoint T-mode CH1	INT32	R/W	Kelvin × 100. Write reg14=hi, reg15=lo
21	Work mode CH1: 0=Manual, 1=Auto, 2=External (RO), 3=PID Out (RO)	UINT8	R/W	Must be 1 (Auto) before PID accepts setpoint
43	PID setpoint ramp CH1	INT16	R/W	× 10
70	Go to OPERATE CH1	UINT8	W	Write 1
71	Go to STANDBY CH1	UINT8	W	Write 1
72	Switch PID to RUN CH1	UINT8	W	Write 1
73	Switch PID to HOLD CH1	UINT8	W	Write 1
81–82	PID actual setpoint CH1 (read-back of current ramped SP)	INT32	R	Kelvin × 100
151	PID output (auto mode) CH1	INT16	R	× 100 = %
248	Output power CH1	INT16	R	× 100 = W

CH2 (Substrate Heater) Registers

Address	Function	Type	R/W	Notes / Values
84	Status CH2 (bit 0 = OPERATE state)	INT16	R	Bit field
87	PID state CH2: 0=Off, 1=Run, 2=Hold	UINT8	R	
88–89	Process value CH2	INT32	R	Kelvin × 100
90–91	PID setpoint T-mode	INT32	R/W	Kelvin × 100

	CH2			
97	Work mode CH2	UINT8	R/W	Same values as reg 21 (CH1)
114	Go to OPERATE CH2	UINT8	W	Write 1
115	Go to STANDBY CH2	UINT8	W	Write 1
116	Switch PID to RUN CH2	UINT8	W	Write 1
117	Switch PID to HOLD CH2	UINT8	W	Write 1
153	PID output (auto mode) CH2	INT16	R	× 100 = %
249	Output power CH2	INT16	R	× 100 = W

Global Device Registers

Address	Function	Type	R/W	Notes / Values
1000	Remote control (MASTER): write 1 = acquire, write 0 = release	UINT8	R/W	Heartbeat: READ this range every ≤10 s to keep MASTER
1151	Remote enable status: 1 = Remote Enable is ON in device menu	UINT8	R	Must be 1 before software control works

11. B-RAX 3400 Full Command Reference

All commands use InstruTech RS-485 protocol (COMM Type = RS485). Format: #AA<CMD><CR> where AA is the two-hex-character device address (default 01). Responses are 13 characters: *AA <value><CR>. Error responses start with ?.

Pressure Reading Commands

Command / TX	Response / RX	Description
#01RDIG<CR>	*01 1.53E-06<CR>	Read ion gauge pressure. Returns 1.10E+03 if IG off or sensor disconnected.
#01RDCG1<CR>	*01 7.60E+02<CR>	Read convectron gauge 1 (roughing range)
#01RDCG2<CR>	*01 7.60E+02<CR>	Read convectron gauge 2
#01RDAI<CR>	*01 7.60E+02<CR>	Read analog input channel
#01RDIGE<CR>	*01 1.00E-04<CR>	Read IG emission current (A). 0 if IG off.
#01RDIGV<CR>	*01 2.30E+00<CR>	Read IG filament voltage (V). 0 if IG off.
#01RDIGA<CR>	*01 3.10E+00<CR>	Read IG filament current (A). 0 if IG off.
#01RDIGC<CR>	*01 4.77E-09<CR>	Read IG ion current (A).

Ion Gauge Control Commands

Command / TX	Response / RX	Description
#01IG1<CR>	*01 PROGM_OK<CR>	Turn IG ON. Error if pressure > 1E-3 Torr or IG error exists.
#01IG0<CR>	*01 PROGM_OK<CR>	Turn IG OFF. Also clears IG error flags.
#01IGS<CR>	*01 1 IG_ON <CR>	Query IG state: 1=ON, 0=OFF
#01RSIG<CR>	*01 00 ST_OK<CR>	IG diagnostic: 00=OK, 01=OVPRS, 02=EMISS, 04=FLVLO, 08=FLOPN, 10=DEGAS, 20=ICLOW, 40=FLIHI
#01DG1<CR>	*01 PROGM_OK<CR>	Start IG degas (requires IG on, P < 5×10^{-5} Torr)
#01DG0<CR>	*01 PROGM_OK<CR>	Stop IG degas
#01DGS<CR>	*01 1 DG_ON <CR>	Query degas state: 1=ON, 0=OFF
#01SE1<CR>	*01 PROGM_OK<CR>	Set emission current to 4 mA (y=1=4mA, y=0=100µA)
#01SF1<CR>	*01 PROGM_OK<CR>	Select filament (y=1=F1, y=2=F2)
#01VER<CR>	*01 01306-11<CR>	Read firmware version

Communication Notes

- All commands start with #, all normal responses start with *
- Maximum command repetition rate: 46 ms at 19200 baud (from manual)
- The _parse_pressure() driver method: strips leading *, skips 3 chars (2-char address + space), converts rest to float. Returns None for ? responses.
- Hardware handshake (RTS/CTS/DTR) is not used by the B-RAX 3400

- If the ion gauge cannot be turned on, check pressure is below interlock threshold (1×10^{-3} Torr at 100 µA emission, 5×10^{-4} at 4 mA) and send #01IG0<CR> first to clear error flags

End of Document